

Primitiv-rekursive Funktionen —Alternative Berechnungsmodelle—

Markus Hecher

Universität Potsdam, Germany

Vorlesung am Apr 27, 2026

Last updated: 2026-04-27

Recap und Vertiefung

Recap und Vertiefung Mächtigkeit von Loop/While/Goto

Wie Mächtigkeit?!

↪ Reduktionen

- Wir reduzieren von Problem A auf B (notiert $A \leq B$; A “leichter-gleich” B) ... und zeigen, dass jede Instanz von A auf eine Instanz von B reduziert werden kann
- In vielen Kontexten ist $\leq$ transitiv (zB. bei Berechenbarkeit)

- $A \leq_R B$ gibt an, dass wir A auf B reduzieren können via Reduktion R

↪ Reduktion $R : A \rightarrow B$ nimmt eine Instanz von A (gesehen als Menge von Instanzen) und retourniert eine Instanz von B

■ Achtung, es gibt Spielregeln!

- Kosten: Um A via B mittels R zu lösen, müssen wir Ressourcen für R (nicht vergessen!) und jene zum Lösen von $R(I)$ für eine Instanz $I \in A$ berücksichtigen!

Beispiel: Es ist falsch zu behaupten, dass A mittels Reduktion $R : A \rightarrow B$ auf B berechnet werden kann, obwohl R nicht einmal berechenbar ist! ↪ Um Berechenbarkeit zu erhalten, dürfen wir nur berechenbare Reduktionen R verwenden

↪ Korrektheit: $I \in A$ ist “äquivalent” (je nach Kontext) zu $f(I) \in B$

Reduktionsrichtung?

↪ Reduktion von Problem A auf B zur Problemlösung:

- “Einfachheit”: zB. Berechenbarkeit, Handhabbarkeit, Effizienz (oder konkret)
- Wir lösen A via B , um zu zeigen, dass A “einfach” ist.
- Wir weisen damit Machbarkeit (evtl. sogar Algorithmus) von A nach

↪ Lösen von A ; direkt via Reduktionsaufwand von R und Lösungsaufwand von B

↪ Reduktion von “hartem” Problem A auf B zur Nichtlösung:

“Härte”: zB. Unberechenbarkeit, Nichthandhabbarkeit, Ineffizienz (oder konkret)

- Wir wissen also (evtl. bedingt, unter einer üblichen Annahme), dass A “hart” ist
- Sollten wir es allerdings schaffen, A besser/schneller via B zu lösen. . .
... dann ist auch B “hart”; die “Härte” überträgt sich daher auf B

↪ Direktvererbung gewisser Nichtmachbarkeitseigenschaften auf B

Konzeptionell funktionieren beide Richtungen gleich, haben nur einen anderen Zweck!

Vergleich zur While-Berechenbarkeit I

Theorem 1.

$$\mathcal{GF} = \mathcal{WF}$$

Beweisskizze

- “ $\supseteq$ ”:

- Sei $f \in \mathcal{WF}$

↪ Daher gibt es $P_W \in \mathcal{WP}$ der Form $P_W = A_1; \dots; A_n$, das diese Funktion berechnet

- Reduktion $R(P_W)$: Konstruiere zunächst $M_1 : A_1; \dots; M_n : A_n; M_{n+1} : \mathbf{halt}$

- Ersetze jedes $M_i : \mathbf{while } x_i \mathbf{ do } P \mathbf{ end}$ durch

$M_i : \mathbf{if } x_i = 0 \mathbf{ goto } M_{i+1};$

$P;$

$M_{\text{new}_i} : \mathbf{goto } M_i;$

$M_{i+1} : \dots$

Vergleich zur While-Berechenbarkeit II

Theorem 1.

$$\mathcal{GF} = \mathcal{WF}$$

Beweisskizze

- “ $\supseteq$ ”:
 - ...Übersetze P rekursiv durch ein äquivalentes $P' \in \mathcal{GP}$, aber lasse dabei den letzten halt-Befehl weg
 - Baue fehlende Marker ein, shifte die Nummerierung der alten Marker entsprechend und passe auch die goto-Befehle an, um $P_G = R(P_W)$ zu erhalten
 - Korrektheit von R : Weise nach, dass $f_{P_G} = f_{P_W} = f$

Vergleich zur While-Berechenbarkeit III

Theorem 1.

$$\mathcal{GF} = \mathcal{WF}$$

Beweisskizze

■ “ $\subseteq$ ”:

■ Sei $f \in \mathcal{GF}$

⇒ Dann gibt es $P_G \in \mathcal{GP}$ mit $P_G = M_1 : A_1; \dots; M_n : A_n$, das f berechnet

■ Reduktion $R'(P_G)$: Konstruktion von P_W mit Extra-Variable y , um Marker zu simulieren, sowie einer (sichtbaren) While-Schleife zur Fallunterscheidung ...

Vergleich zur While-Berechenbarkeit IV

Theorem 1.

$$\mathcal{GF} = \mathcal{WF}$$

Beweisskizze

■ “ $\subseteq$ ”: ...

```
y := 1;  
while y  $\neq$  0 do  
  if y = 1 then  
    ... //  $A_1$   
  end;  
  ...  
  if y = n then  
    ... //  $A_n$   
  end  
end
```

Vergleich zur While-Berechenbarkeit V

Theorem 1.

$$\mathcal{GF} = \mathcal{WF}$$

Beweisskizze

- “ $\subseteq$ ”:

- ...Übersetze $M_k : A_k$:

- $M_k : x_i := 0$ durch $x_i := 0; y := k + 1$
- $M_k : x_i := \text{succ}(x_j)$ durch $x_i := \text{succ}(x_j); y := k + 1$
- $M_k : x_i := \text{pred}(x_j)$ durch $x_i := \text{pred}(x_j); y := k + 1$
- $M_k : \text{if } x_i \text{ goto } M_l$ durch $y := k + 1; \text{if } x_i \text{ then } y := l \text{ end}$
- $M_k : \text{halt}$ durch $y := 0$

- Korrektheit von R' : Weise nach, dass $f_{P_W} = f_{P_G} = f$

Nebenergebnis und Korollar

Theorem 2 (Satz von Kleene).

Jede While-berechenbare Funktion lässt sich – abgesehen von Loop-Simulationen – durch ein While-Programm mit nur einer While-Schleife berechnen

Korollar.

$$\mathcal{TGF} = \mathcal{TWF}$$

Vergleich zur Turing-Berechenbarkeit I

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

- $\mathcal{WF} \subseteq \mathcal{TF}$:

- Sei $f \in \mathcal{WF}$

- ↪ Dann gibt es P_W , das dieses berechnet

- Reduktion $R_T(P'_W)$ via $P'_W = R'(R(P_W))$:

- ↪ Wandle P_W gemäß der Konstruktion von Theorem 2 zunächst in P_G und dann in P'_W um; sodass P'_W von der folgenden Form ist (A_i ohne echte While-Schleifen) ...

Vergleich zur Turing-Berechenbarkeit II

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

■ “ $\subseteq$ ”: ...

```
y := 1;  
while y  $\neq$  0 do  
  if y = 1 then  
    ... //  $A_1$   
  end;  
  ...  
  if y = n then  
    ... //  $A_n$   
  end  
end
```

Vergleich zur Turing-Berechenbarkeit III

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

■ $\mathcal{WF} \subseteq \mathcal{TF}$: ...

- Simulation mit TM M mit $I_{P'_W} + 1$ Bändern: eins für jede Variable und ein Band zum Herunterzählen für Loop-Schleifen
- Konstruktion von Elementarmaschinen zum Nullsetzen ($\text{Init}(x_i)$), Kopieren ($\text{Copy}(x_i, x_j)$), bilden des Vorgängers ($\text{Pred}(x_i)$) und des Nachfolgers ($\text{Suc}(x_i)$)

⇒ Aufbauend darauf lassen sich TMs konstruieren für die Hintereinanderausführung sowie Loop-Schleifen (mittels des Hilfsbandes)

- D.h. für jedes A_i lässt sich eine Maschine M_i konstruieren

Vergleich zur Turing-Berechenbarkeit IV

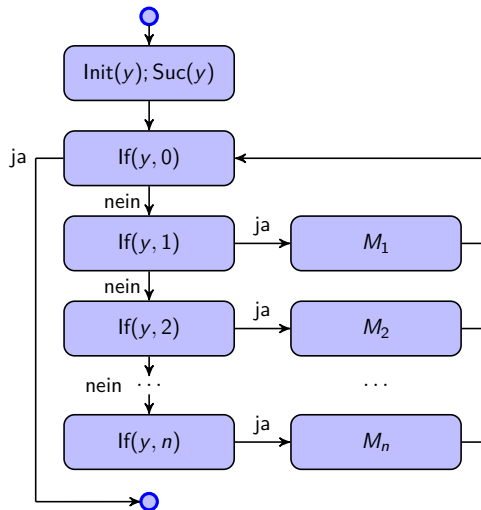
Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

- $\mathcal{WF} \subseteq \mathcal{TF}$:
 - ...Es lässt sich für jedes n eine Maschine $\text{If}(y, n)$ mit einem Ja- und einem Nein-Ausgang konstruieren, die checkt, ob $y = n$ ist
 - Es lässt sich nun $M = R_T(P'_W)$ wie auf der nächsten Folie dargestellt konstruieren
 - Korrektheit von R_T : Es verbleibt eigentlich zu zeigen, dass $f_M = f_{P'_W} = f$

Vergleich zur Turing-Berechenbarkeit V



Vergleich zur Turing-Berechenbarkeit VI

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

- $\mathcal{TF} \subseteq \mathcal{GF}$:

- Sei $f \in \mathcal{TF}$

- ↪ Es gibt eine zugehörige unäre Funktion $f' : \subseteq \{1\}^* \rightarrow \{1\}^*$ mit $f' \in \mathcal{TF}_{\{1\}}$

- Und damit wiederum eine TM $T = (Q, \{1\}, \Gamma, \delta, q_0, B, F)$, die f' berechnet

- OBdA seien $Q = \{0, \dots, n-1\}$, $\Gamma = \{0, \dots, m-1\}$ ($m \geq 2$, da $1, B \in \Gamma$) und $B=0$

- Reduktion R_G : Grundidee der Konstruktion von P_G ist Codierung von Konfigurationen und Simulation der Konfigurationsübergänge

Vergleich zur Turing-Berechenbarkeit VII

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

- $\mathcal{TF} \subseteq \mathcal{GF}$:

- ...Konfiguration (α, q, β) bei $\alpha = a_{\ell-1} \dots a_0$ und $\beta = b_0 \dots b_{k-1}$ durch

- $x_q := q$

- $x_\alpha := \sum_{i=0}^{\ell-1} a_i \cdot m^i$

- $x_\beta := \sum_{i=0}^{k-1} b_i \cdot m^i$

↪ x_α und x_β m -adische Darstellungen (wegen $m \geq 2$ eindeutig)

Vergleich zur Turing-Berechenbarkeit VIII

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

■ $\mathcal{TF} \subseteq \mathcal{GF}$:

■ ...Konstruiere P_{Gij} für alle i, j mit $\delta(i, j) \neq \perp$

■ Fall $\delta(i, j) = (i', j', L)$:

$x_\beta := x_\beta \text{ div } m;$ /* Zeichen über LS-Kopf

$x_\beta := j' + m \cdot x_\beta;$ überschreiben */

$x_\beta := x_\alpha \text{ mod } m + m \cdot x_\beta;$ /* LS-Kopf

$x_\alpha := x_\alpha \text{ div } m;$ schieben */

$x_q := i'$ // Folgezustand

Vergleich zur Turing-Berechenbarkeit IX

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

■ $\mathcal{TF} \subseteq \mathcal{GF}$:

■ Konstruiere $P_{G_{ij}}$ für alle i, j mit $\underline{\delta(i, j) \neq \perp}$

■ Fall $\delta(i, j) = (i', j', R)$:

$x_\beta := x_\beta \text{ div } m;$ /* Zeichen über LS-Kopf

$x_\beta := j' + m \cdot x_\beta;$ überschreiben */

$x_\alpha := x_\beta \text{ mod } m + m \cdot x_\alpha;$ /* LS-Kopf

$x_\beta := x_\beta \text{ div } m;$ schieben */

$x_q := i'$ // Folgezustand

Vergleich zur Turing-Berechenbarkeit X

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

■ $\mathcal{TF} \subseteq \mathcal{GF}$: ...

■ Konstruiere Hauptteil von P_G (für $\delta(i,j) \neq \perp$; simuliert Konfigurationsübergänge):

$M_2 : x_a := x_\beta \bmod m$;

if $(x_q = 1 \wedge x_a = 1)$ **goto** M_{11} ;

if $(x_q = 1 \wedge x_a = 2)$ **goto** M_{12} ;

...

if $(x_q = 2 \wedge x_a = 1)$ **goto** M_{21} ;

if $(x_q = 2 \wedge x_a = 3)$ **goto** M_{23} ; // Hier: $\delta(2,2) = \perp$

...

if $(x_q \in F \wedge x_\alpha = 0)$ **goto** M_3 **else goto** M_2 ; // $\alpha = B^\ell$

Vergleich zur Turing-Berechenbarkeit XI

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

■ $\mathcal{TF} \subseteq \mathcal{GF}$: ...

■ Konstruiere Hauptteil von P_G (für $\underline{\delta(i,j)} \neq \perp$; simuliert Konfigurationsübergänge):

...

$M_{11} : P_{G_{11}}; \textbf{goto } M_2;$

$M_{12} : P_{G_{12}}; \textbf{goto } M_2;$

...

$M_{21} : P_{G_{21}}; \textbf{goto } M_2;$

$M_{23} : P_{G_{23}}; \textbf{goto } M_2;$

...

Vergleich zur Turing-Berechenbarkeit XII

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

- $\mathcal{TF} \subseteq \mathcal{GF}$: ...
 - P_G wird wie folgt konstruiert. Zuerst muss P_G Variablen initialisieren:
 $M_1 : x_\alpha := 0;$
 $x_q := q_0;$
 $x_\beta := \sum_{i=0}^{|Eingabe|-1} Eingabe_i \cdot m^i$
 - Dann obiger Hauptteil beginnend mit Marker M_2
 - Schließlich fehlt noch Marker M_3 (Terminierung): Decodierung von x_β und Testen, ob alle b_i – wie gefordert – 1en sind, bis nur noch Blanks (0) kommen
- ⇒ Korrektheit von R_G : Eigentlich fehlt noch Korrektheitsnachweis der Konstruktion

Korollar für totale Berechenbarkeitsmodelle

Korollar.

$$(\mathcal{TGF} = \mathcal{TWF}) = \mathcal{TF}$$

Zusammenfassung

- TMs (Symbole schieben) vs. Loop/While/Goto (simple Arithmetik + Kontrollstr.)
 - Resultate: $\mathcal{LF} \subseteq (\mathcal{TGF} = \mathcal{TWF} = \mathcal{TF}) \subset (\mathcal{GF} = \mathcal{WF} = \mathcal{F})$
 - Gilt auch $(\mathcal{TGF} = \mathcal{TWF} = \mathcal{TF}) \subseteq \mathcal{LF}$?
- ↪ Treffen Berechenbarkeitsmodelle exakt die (total) berechenbaren Funktionen?

Zusammenfassung: Reduktionen!

- Grundkonzept, um Problem A in Problem B umzuwandeln (A leichtergleich B)
- Wichtige Bestandteile
 - Kosten: Kontextabhängig; sollte die Kosten von B nicht überschreiten
 - Korrektheit: Kontextabhängige Äquivalenz nachweisen
- ↪ Richtung ist entscheidend!
 - Reduktion von hartem Problem A auf B : B ist mindestens so hart wie A
 - Reduktion von A auf einfaches Problem B : A mindestens so einfach wie B
- ↪ Zweck/Ziel bestimmt sozusagen die Richtung

Primitiv-rekursive Funktionen

Motivation

- Mathematischer, von Maschinen unabhängiger Ansatz
 - Abstrakt; Berechenbarkeitsmodell direkt auf semantischer Ebene
 - Charakterisierung totaler Berechenbarkeit als Ziel
- ~> Mächtigkeitsvergleiche zu $\mathcal{LF}$ sowie $\mathcal{TF} = \mathcal{GF} = \mathcal{WF}$

Primitiv-rekursive Funktionen

Das Berechenbarkeitsmodell

Grundfunktionen

- Nachfolgerfunktion $\text{suc} : \mathbb{N} \rightarrow \mathbb{N}$
 $\text{suc}(n) = n + 1$
- Projektionsfunktionen $\text{pr}_i^\ell : \mathbb{N}^\ell \rightarrow \mathbb{N}$ für $1 \leq i \leq \ell$
 $\text{pr}_i^\ell(n_1, \dots, n_\ell) = n_i$
- Einstellige Nullfunktion $\text{const}_0^1 : \mathbb{N} \rightarrow \mathbb{N}$
 $\text{const}_0^1(n) = 0$
- Nullstellige Nullfunktion (Nullkonstante) $\text{const}_0^0 : \mathbb{N}^0 \rightarrow \mathbb{N}$
 $\text{const}_0^0() = 0$

Primitive Rekursion erhaltende Operationen

- Komposition $f \circ (g_1, \dots, g_\ell) : \mathbb{N}^k \rightarrow \mathbb{N}$
(wobei $f : \mathbb{N}^\ell \rightarrow \mathbb{N}$ und $g_1, \dots, g_\ell : \mathbb{N}^k \rightarrow \mathbb{N}$)
$$f \circ (g_1, \dots, g_\ell)(n_1, \dots, n_k) = f(g_1(n_1, \dots, n_k) , \dots , g_\ell(n_1, \dots, n_k))$$
- Primitive Rekursion $\text{Pr}[f, g] : \mathbb{N}^{\ell+1} \rightarrow \mathbb{N}$
(wobei $f : \mathbb{N}^\ell \rightarrow \mathbb{N}$ und $g : \mathbb{N}^{\ell+2} \rightarrow \mathbb{N}$)
$$\text{Pr}[f, g](n_1, \dots, n_\ell, 0) = f(n_1, \dots, n_\ell) \text{ und}$$
$$\text{Pr}[f, g](n_1, \dots, n_\ell, m+1) = g(n_1 , \dots , n_\ell , m , \text{Pr}[f, g](n_1, \dots, n_\ell, m))$$

Primitiv-rekursive Berechenbarkeit

- Berechenbarkeitsbegriff fällt mit Funktionsbegriff zusammen:
Funktion $f : \mathbb{N}^\ell \rightarrow \mathbb{N}$ heißt primitiv-rekursiv berechenbar gdw. sie primitiv-rekursiv ist
- $\mathcal{PF}$ = Menge aller primitiv-rekursiven Funktionen

Primitiv-rekursive Funktionen

Beispiele

Konstantenfunktionen

- Für alle $k \in \mathbb{N}$ ist $\text{const}_k^1 : \mathbb{N} \rightarrow \mathbb{N}$ mit $\text{const}_k^1(n) = k$ primitiv-rekursiv
 - Beweis durch Induktion über k
 - IA: const_0^1 ist eine Grundfunktion und daher primitiv-rekursiv
 - IS: Sei const_k^1 primitiv-rekursiv. Dann ist $\text{const}_{k+1}^1 = \text{succ} \circ \text{const}_k^1$ primitiv-rekursiv (Komposition primitiv-rekursiver Funktionen)
- Analog $\text{const}_k^0 \in \mathcal{PF}$
- Für alle $k, \ell \in \mathbb{N}$ ist $\text{const}_k^\ell : \mathbb{N}^\ell \rightarrow \mathbb{N}$ mit $\text{const}_k^\ell(n_1, \dots, n_\ell) = k$ primitiv-rekursiv
 - $\text{const}_k^\ell = \text{const}_k^1 \circ \text{pr}_1^\ell$ für $\ell \geq 2$

Vorgängerfunktion

- Im Gegensatz zur Nachfolgerfunktion keine Grundfunktion

- $\text{pred} : \mathbb{N} \rightarrow \mathbb{N}$ rekursiv definiert durch

$$\text{pred}(m) = \begin{cases} m - 1 & \text{falls } m \geq 1, \\ 0 & \text{falls } m = 0 \end{cases}$$

- $\text{pred} = \text{Pr}[f, g]$ angestrebt mit $f : \mathbb{N}^0 \rightarrow \mathbb{N}$ und $g : \mathbb{N}^2 \rightarrow \mathbb{N}$

- Notwendig: $0 = \text{pred}(0) = f()$ und $m = \text{pred}(m + 1) = g(m, \text{pred}(m))$

⇒ **pred** = $\text{Pr}[\text{const}_0^0, \text{pr}_1^2]$, also $\text{pred} \in \mathcal{PF}$

Primitiv-rekursiven Ausdruck analysieren I

■ Betrachte $f = \text{Pr}[\text{pr}_1^1, \text{suc} \circ \text{pr}_3^3]$

■ $f(n, 0) = \text{pr}_1^1(n) = n$

■ $f(n, m+1) = \text{suc} \circ \text{pr}_3^3(n, m, f(n, m)) = \text{suc}(\text{pr}_3^3(n, m, f(n, m))) = \text{suc}(f(n, m)) = f(n, m) + 1$

$$\text{ZB. } f(2, 3) = f(2, 2) + 1 = f(2, 1) + 1 + 1 = f(2, 0) + 1 + 1 + 1 = 2 + \underbrace{1 + 1 + 1}_{\text{3-mal}}$$

↪ f ist die Addition: **add** $\in \mathcal{PF}$

Primitiv-rekursiven Ausdruck analysieren II

- Betrachte $f = \text{Pr}[\text{const}_0^0, \text{const}_1^2]$
 - $f(0) = \text{const}_0^0() = 0$
 - $f(m+1) = \text{const}_1^2(m, f(m)) = 1$
 - $f(m) = \begin{cases} 1 & \text{falls } m \geq 1, \\ 0 & \text{falls } m = 0 \end{cases}$

↪ f ist die Signums-Funktion: **sign** $\in \mathcal{PF}$

Subtraktion

- $\text{sub} : \mathbb{N}^2 \rightarrow \mathbb{N}$ rekursiv definiert durch
$$\text{sub}(n, m) = \begin{cases} n & \text{falls } m = 0, \\ \text{pred}(\text{sub}(n, m - 1)) & \text{falls } m \geq 1 \end{cases}$$
 - $\text{sub} = \text{Pr}[f, g]$ angestrebt mit $f : \mathbb{N} \rightarrow \mathbb{N}$ und $g : \mathbb{N}^3 \rightarrow \mathbb{N}$
 - Notwendig: $n = \text{sub}(n, 0) = f(n)$ und
$$\text{sub}(n, m + 1) = \text{pred}(\text{sub}(n, m)) = g(n, m, \text{sub}(n, m))$$
 - Realisierbar durch $f = \text{pr}_1^1$ und $g = \text{pred} \circ \text{pr}_3^3$
- ↪ Also **sub** = $\text{Pr}[\text{pr}_1^1, \text{pred} \circ \text{pr}_3^3]$ und $\text{sub} \in \mathcal{PF}$

Primitiv-rekursive Funktionen

Primitiv-rekursive Aussagefunktionen

Kleiner Gleich-Relation

- Sei $f, g : \mathbb{N}^\ell \rightarrow \mathbb{N} \in \mathcal{PF}$
- Dann ist $f \leq g : \mathbb{N}^\ell \rightarrow \{0, 1\}$ primitiv-rekursiv, sodass $f \leq g(n_1, \dots, n_\ell) = 1$ gdw.
 $f(n_1, \dots, n_\ell) \leq g(n_1, \dots, n_\ell)$
 - $\rightsquigarrow f \leq g(n_1, \dots, n_\ell) = 1 - (f(n_1, \dots, n_\ell) - g(n_1, \dots, n_\ell))$
 - Ist $f(n_1, \dots, n_\ell) \leq g(n_1, \dots, n_\ell)$, so ist $f \leq g(n_1, \dots, n_\ell) = 1$
 - Ist $f(n_1, \dots, n_\ell) > g(n_1, \dots, n_\ell)$, so ist $f \leq g(n_1, \dots, n_\ell) = 0$

Logische Konnektive

- Sei $f, g : \mathbb{N}^\ell \rightarrow \mathbb{N} \in \mathcal{PF}$
- $\wedge \in \mathcal{PF}$ wegen $f \wedge g = \text{sign} \circ \text{mul} \circ (f, g)$ ($\text{mul} \in \mathcal{PF}$ in Übung)
- $\neg \in \mathcal{PF}$ wegen $\neg f = \text{sub} \circ (\text{const}_1^\ell, f)$
- $\vee \in \mathcal{PF}$ wegen $f \vee g = \text{sign} \circ \text{add} \circ (f, g)$

Weitere Vergleichsrelationen

- Sei $f, g : \mathbb{N}^\ell \rightarrow \mathbb{N} \in \mathcal{PF}$
- $= \in \mathcal{PF}$ weil $(f=g)(n_1, \dots, n_\ell)$ ist 1
gdw. $(f(n_1, \dots, n_\ell) \leq g(n_1, \dots, n_\ell)) \wedge (g(n_1, \dots, n_\ell) \leq f(n_1, \dots, n_\ell))$
- $\neq \in \mathcal{PF}$ weil $(f \neq g)(n_1, \dots, n_\ell)$ ist 1
gdw. $\neg(f(n_1, \dots, n_\ell) = g(n_1, \dots, n_\ell))$
- $< \in \mathcal{PF}$ weil $(f < g)(n_1, \dots, n_\ell)$ ist 1
gdw. $(f(n_1, \dots, n_\ell) \leq g(n_1, \dots, n_\ell)) \wedge (f(n_1, \dots, n_\ell) \neq g(n_1, \dots, n_\ell))$

Primitiv-rekursive Funktionen Makros

Fallunterscheidung

$$\blacksquare f, g, \varphi : \mathbb{N}^\ell \rightarrow \mathbb{N} \in \mathcal{PF}$$

$\rightsquigarrow \text{Cond}[\varphi, f, g] : \mathbb{N}^\ell \rightarrow \mathbb{N}$ in $\mathcal{PF}$, mit

$$\text{Cond}[\varphi, f, g](n_1, \dots, n_\ell) = \begin{cases} f(n_1, \dots, n_\ell) & \text{falls } \varphi(n_1, \dots, n_\ell) > 0, \\ g(n_1, \dots, n_\ell) & \text{sonst} \end{cases}$$

$$\blacksquare \text{Cond}[\varphi, f, g](n_1, \dots, n_\ell) = f(n_1, \dots, n_\ell) \cdot \varphi(n_1, \dots, n_\ell) + g(n_1, \dots, n_\ell) \cdot (1 - \varphi(n_1, \dots, n_\ell))$$

$$\blacksquare \text{Cond}[\varphi, f, g] = \text{add} \circ (\text{mul} \circ (f, \varphi), \text{mul} \circ (g, \text{sub} \circ (\text{const}_1^\ell, \varphi)))$$

Beschränkte Minimierung I

$$\blacksquare \varphi : \mathbb{N}^{\ell+1} \rightarrow \mathbb{N} \in \mathcal{PF}$$

$\rightsquigarrow \text{Mn}[\varphi] : \mathbb{N}^{\ell+1} \rightarrow \mathbb{N}$ in $\mathcal{PF}$, mit

$$\text{Mn}[\varphi](n_1, \dots, n_\ell, n) = \begin{cases} \min\{m \leq n \mid \varphi(n_1, \dots, n_\ell, m) > 0\} & \text{falls dies existiert,} \\ n + 1 & \text{sonst} \end{cases}$$

$$\blacksquare \text{Daher: } \text{Mn}[\varphi](n_1, \dots, n_\ell, 0) = \begin{cases} 0 & \text{falls } \varphi(n_1, \dots, n_\ell, 0) > 0, \\ 1 & \text{sonst} \end{cases}$$

$$\blacksquare \text{Mn}[\varphi](n_1, \dots, n_\ell, n + 1) = \begin{cases} \text{Mn}[\varphi](n_1, \dots, n_\ell, n) & \text{falls } \text{Mn}[\varphi](n_1, \dots, n_\ell, n) \leq n, \\ n + 1 & \text{falls } \text{Mn}[\varphi](n_1, \dots, n_\ell, n) = n + 1, \\ & \varphi(n_1, \dots, n_\ell, n + 1), \\ n + 2 & \text{sonst} \end{cases}$$

Beschränkte Minimierung II

- Zuerst:
$$\text{Mn}[\varphi](n_1, \dots, n_\ell, 0) = \begin{cases} 0 & \text{falls } \varphi(n_1, \dots, n_\ell, 0) > 0, \\ 1 & \text{sonst} \end{cases} =$$
$$\text{Cond}[\varphi', \text{const}_0^\ell, \text{const}_1^\ell](n_1, \dots, n_\ell) \quad \text{mit } \varphi' = \varphi \circ (\text{pr}_1^\ell, \dots, \text{pr}_\ell^\ell, \text{const}_0^\ell)$$

Beschränkte Minimierung III

- Fehlt noch:

$$\text{Mn}[\varphi](n_1, \dots, n_\ell, n+1) = \begin{cases} \text{Mn}[\varphi](n_1, \dots, n_\ell, n) & \text{falls } \text{Mn}[\varphi](n_1, \dots, n_\ell, n) \leq n, \\ n+1 & \text{falls } \text{Mn}[\varphi](n_1, \dots, n_\ell, n) = n+1 \wedge \\ & \varphi(n_1, \dots, n_\ell, n+1), \\ n+2 & \text{sonst} \end{cases}$$

$$\rightsquigarrow \text{Mn}[\varphi](n_1, \dots, n_\ell, n+1) = \text{Cond}[\psi, \text{pr}_{\ell+2}^{\ell+2}, \text{Cond}[\chi, \text{suc} \circ \text{pr}_{\ell+1}^{\ell+2}, \text{suc} \circ \text{suc} \circ \text{pr}_{\ell+1}^{\ell+2}]](n_1, \dots, n_\ell, n, \text{Mn}[\varphi](n_1, \dots, n_\ell, n))$$

- Weil $\psi, \chi : \mathbb{N}^{\ell+2} \rightarrow \mathbb{N}$ primitiv-rekursiv:

- mit $\psi(n_1, \dots, n_\ell, n, p) = p \leq n$
- mit $\chi(n_1, \dots, n_\ell, n, p) = (p = n+1 \wedge \varphi(n_1, \dots, n_\ell, n+1))$

Beschränkte Minimierung IV

- Wir haben $\text{Mn}[\varphi](n_1, \dots, n_\ell, 0) = \text{Cond}[\varphi', \text{const}_0^\ell, \text{const}_1^\ell](n_1, \dots, n_\ell)$
- Außerdem $\text{Mn}[\varphi](n_1, \dots, n_\ell, n+1) =$
 $\text{Cond}[\psi, \text{pr}_{\ell+2}^{\ell+2}, \text{Cond}[\chi, \text{suc} \circ \text{pr}_{\ell+1}^{\ell+2}, \text{suc} \circ \text{suc} \circ \text{pr}_{\ell+1}^{\ell+2}]]$
 $(n_1, \dots, n_\ell, n, \text{Mn}[\varphi](n_1, \dots, n_\ell, n))$

$$\rightsquigarrow \text{Mn}[\varphi] = \text{Pr}[\text{Cond}[\varphi', \text{const}_0^\ell, \text{const}_1^\ell], \text{Cond}[\psi, \text{pr}_{\ell+2}^{\ell+2}, \text{Cond}[\chi, \text{suc} \circ \text{pr}_{\ell+1}^{\ell+2}, \text{suc} \circ \text{suc} \circ \text{pr}_{\ell+1}^{\ell+2}]]]$$

$$\rightsquigarrow \text{Mn}[\varphi] \in \mathcal{PF}$$

- Beschränkte Maximierung $\text{Mx}[\varphi] \in \mathcal{PF}$ analog

Cantorsche Paarungsfunktion

Cantorsche Paarungsfunktion

Motivation und Definition

Warum?

- Wir wollen schlussendlich (zumindest) $\mathcal{LF}$ durch $\mathcal{PF}$ simulieren
- Funktionen in $\mathcal{PF}$ sind aber von der Form $\mathbb{N}^\ell \rightarrow \mathbb{N}$
- ↪ Ein ganzer Speichervektor (in $\mathbb{N}^{\ell'}$) muss daher irgendwie als Zahl (in $\mathbb{N}$) repräsentiert werden
- Wie? Cantorsche Paarungsfunktion: wandle Vektor in eine Zahl um

Definition

⋮	⋮	⋮	⋮	⋮	⋮	⋮	...
5	20						...
4	14	19					...
3	9	13	18				...
2	5	8	12	17			...
1	2	4	7	11	16		...
0	0	1	3	6	10	15	...
m/n	0	1	2	3	4	5	...

- $\langle 0, 0 \rangle = 0$
- $\langle n + 1, 0 \rangle = \langle n, 0 \rangle + (\text{"Länge der Diagonale bei } \langle n, 0 \rangle \text{ beginnend"} - 1) + 1 = \langle n, 0 \rangle + n + 1$
- $\langle n, m + 1 \rangle = \langle n + 1, m \rangle + 1$

$$\langle \rangle : \mathbb{N}^2 \rightarrow \mathbb{N} \text{ mit}$$

$$\langle n, m \rangle = \left(\sum_{i=0}^{n+m} i \right) + m = \frac{(n+m) * (n+m+1)}{2} + m$$

Erweiterung auf beliebige Pseudotupel

- $\langle \rangle^\ell : \mathbb{N}^\ell \rightarrow \mathbb{N}, \ell \geq 1$ mit ...
- $\langle n \rangle^1 = n$ (Basisfall)
- $\langle n_1, \dots, n_{\ell+1} \rangle^{\ell+1} = \langle \langle n_1, \dots, n_\ell \rangle^\ell, n_{\ell+1} \rangle$

↪ Kurzschreibweise: $\langle n_1, \dots, n_\ell \rangle$

Cantorsche Paarungsfunktion

Eigenschaften

Eigenschaften

- $\langle \rangle$ bijektiv, primitiv-rekursiv
- $\langle \rangle^\ell$ für $\ell \neq 0$ bijektiv, primitiv-rekursiv
- Umkehrfunktionen $\pi_i^2 = \text{pr}_i^2 \circ \langle \rangle^{-1}$ ebenfalls primitiv-rekursiv
- Umkehrfunktionen $\pi_i^\ell = \text{pr}_i^\ell \circ (\langle \rangle^\ell)^{-1}$ ebenfalls primitiv-rekursiv

⟨⟩ primitiv-rekursiv

- $\langle n, m \rangle = (n + m) \cdot \frac{(n+m+1)}{2} + m$
- Informell: als Kombination von Konstanten, Projektionen, Addition, Multiplikation und Division selbst wieder primitiv-rekursiv
- Formal: $\langle \rangle = \text{add} \circ (\text{div} \circ (\text{mul} \circ (\text{add}, \text{suc} \circ \text{add}), \text{const}_2^2), \text{pr}_1^2)$

$\langle \rangle^\ell$ primitiv-rekursiv

- IA $\ell = 1$:
 - $\langle \rangle^1 = \text{id} = \text{pr}_1^1 \in \mathcal{PF}$
- IS: $\langle \rangle^{\ell+1} = \langle \rangle \circ (\langle \rangle^\ell \circ (\text{pr}_1^{\ell+1}, \dots, \text{pr}_\ell^{\ell+1}), \text{pr}_{\ell+1}^{\ell+1}) \in \mathcal{PF}$

Cantorsche Paarungsfunktion Umkehrfunktionen

Umkehrfunktionen π_i^2 primitiv-rekursiv I

- $\langle \rangle^{-1} : \mathbb{N} \rightarrow \mathbb{N}^2, \quad \pi_i^2 : \mathbb{N} \rightarrow \mathbb{N}$
- Klar: $d = \langle n, m \rangle$ für gewisse $n, m \in \mathbb{N}$, aber nicht, ob diese auch primitiv-rekursiv berechenbar sind
- Zunächst Auffinden des Diagonalwertes $p (=n + m)$:
 $h : \mathbb{N} \rightarrow \mathbb{N}$ mit $h(d) = \min\{p \mid (p + 1) \cdot \frac{p+2}{2} > d\}$

Umkehrfunktionen π_i^2 primitiv-rekursiv II

- Gesucht $h : \mathbb{N} \rightarrow \mathbb{N}$ mit $h(d) = \min\{p \mid (p+1) \cdot \frac{p+2}{2} > d\}$

↪ $h \in \mathcal{PF}$

- Definiere $\varphi : \mathbb{N}^2 \rightarrow \mathbb{N}$ mit $\varphi(d, p) = (p+1) \cdot \frac{p+2}{2} > d$

- Daher $\text{Mn}[\varphi] : \mathbb{N}^2 \rightarrow \mathbb{N} \in \mathcal{PF}$

↪
$$\text{Mn}[\varphi](d, q) = \begin{cases} \min\{p \leq q \mid (p+1) \cdot \frac{p+2}{2} > d\} & \text{falls Minimum existiert} \\ q+1 & \text{sonst} \end{cases}$$

- Insbesondere: $\text{Mn}[\varphi](d, d) = \min\{p \mid (p+1) \cdot \frac{p+2}{2} > d\}$

↪
$$h = \text{Mn}[\varphi] \circ (\text{pr}_1^1, \text{pr}_1^1)$$

Umkehrfunktionen π_i^2 primitiv-rekursiv III

$$\blacksquare \pi_2^2(d) = m = d - (n + m) \cdot \frac{n+m+1}{2} = d - h(d) \cdot \frac{h(d)+1}{2}$$

$$\rightsquigarrow \pi_2^2 = \text{sub} \circ (\text{pr}_1^1, \text{div} \circ (\text{mul} \circ (h, \text{suc} \circ h), \text{const}_2^1)) \in \mathcal{PF}$$

$$\blacksquare \pi_1^2(d) = n = h(d) - m = h(d) - \pi_2^2(d)$$

$$\rightsquigarrow \pi_1^2 = \text{sub} \circ (h, \pi_2^2)$$

Funktionsweise von Umkehrfunktionen π_i^ℓ

- Umkehrfunktionen können für $\langle \rangle^\ell$ erweitert werden

- $$\begin{aligned}\pi_1^\ell(\langle n_1, \dots, n_\ell \rangle^\ell) &= \\ \text{pr}_1^\ell \circ (\langle \rangle^\ell)^{-1}(\langle n_1, \dots, n_\ell \rangle^\ell) &= \\ \text{pr}_1^\ell(n_1, \dots, n_\ell) &= \\ n_1\end{aligned}$$

~> Man kann zeigen, dass auch π_i^ℓ primitiv-rekursiv ist

Cantorsche Paarungsfunktion

Beschränkung auf eine Eingabe

Theorem

Theorem 4.

Für jedes $f : \mathbb{N}^\ell \rightarrow \mathbb{N} \in \mathcal{PF}$ existiert ein $f' : \mathbb{N} \rightarrow \mathbb{N} \in \mathcal{PF}$ mit $f = f' \circ \langle \rangle^\ell$

$$\blacksquare f' = f \circ (\pi_1^\ell, \dots, \pi_\ell^\ell)$$

$$\rightsquigarrow f' \circ \langle \rangle^\ell (n_1, \dots, n_\ell) = \{\text{Def } \circ\}$$

$$f'(\langle n_1, \dots, n_\ell \rangle^\ell) = \{\text{Def } f'\}$$

$$f \circ (\pi_1^\ell, \dots, \pi_\ell^\ell) (\langle n_1, \dots, n_\ell \rangle^\ell) = \{\text{Def } \circ\}$$

$$f(\pi_1^\ell(\langle n_1, \dots, n_\ell \rangle^\ell), \dots, \pi_\ell^\ell(\langle n_1, \dots, n_\ell \rangle^\ell)) = \{\text{Def } \pi_i^\ell + \langle \rangle^\ell\}$$

$$f(n_1, \dots, n_\ell)$$

Quintessenz

Theorem 5.

Für jedes $f : \mathbb{N}^\ell \rightarrow \mathbb{N} \in \mathcal{PF}$ existiert ein $f' : \mathbb{N} \rightarrow \mathbb{N} \in \mathcal{PF}$ mit $f = f' \circ \langle \rangle^\ell$

- Simulation über einstellige primitiv-rekursive Funktionen stets möglich
- Dafür (nur) wesentlich, dass $\langle \rangle^\ell$ und Umkehrfunktionen im Berechenbarkeitsmodell enthalten
- ↪ In jedem Berechenbarkeitsmodell, dessen berechenbare Funktionen $\mathcal{PF}$ umfassen, reicht es also, sich auf einstellige Funktionen zu beschränken